



RETROFIT



Retrofit is a popular Android library for making HTTP requests and simplifying REST API integration by converting HTTP responses into native objects.

It's widely used for network communication in Android apps.



○ API



An API (Application Programming Interface) is defined as a set of protocols that enables the access to external services within our own applications.

Retrofit uses annotations and interfaces to define these API endpoints and their expected responses.

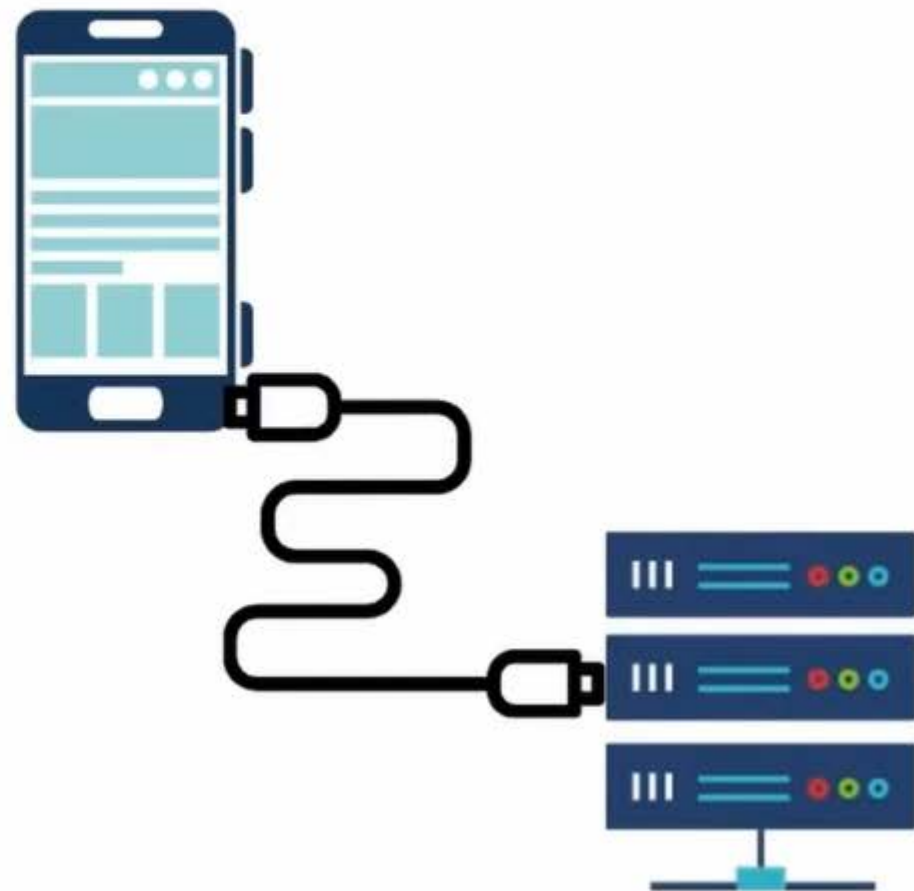


○ REST API



A REST API (Representational State Transfer Application Programming Interface) is a set of HTTP-based endpoints that follows REST principles, allowing for the exchange of data between an app and a remote server,

It uses standard HTTP methods like GET, POST, PUT, and DELETE.

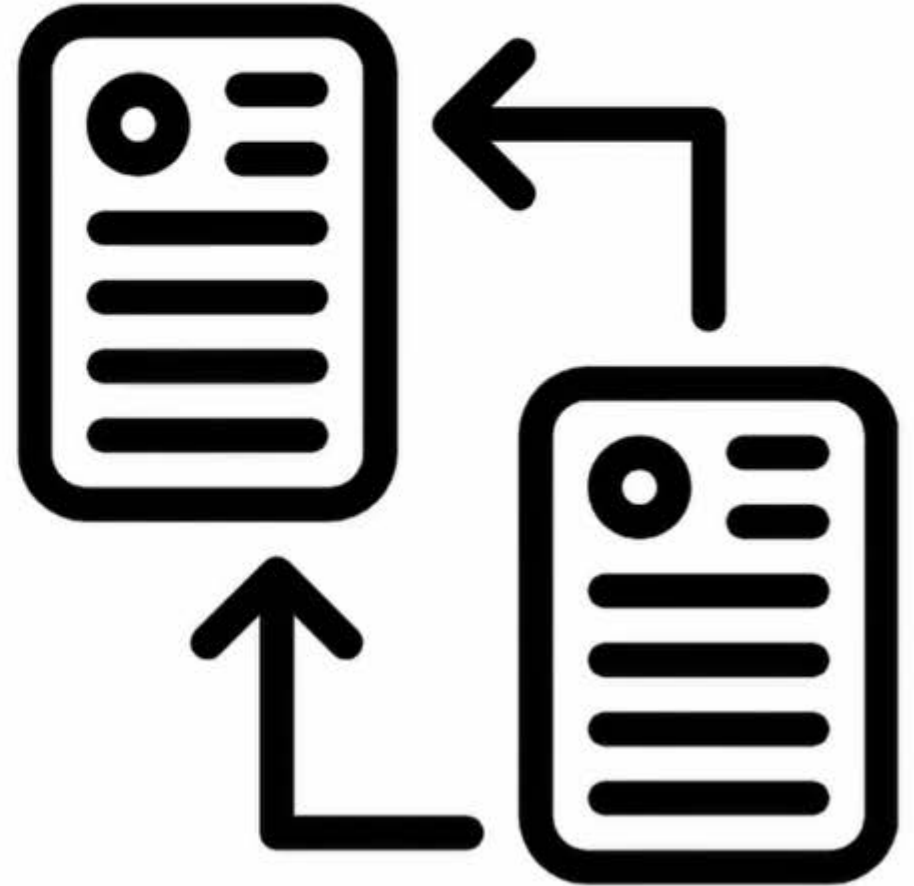




HTTP ANNOTATIONS



- **@GET:** Used to make a GET request to retrieve data.
- **@POST:** Used to make a POST request to send data to the server.
- **@Query:** Used to specify query parameters in the URL. ➔



○ JSON



JSON (JavaScript Object Notation) is a lightweight data interchange format used to structure and transmit data between the app and the RESTful API.

Retrofit can automatically serialize and deserialize JSON data, making it a common choice for data exchange in mobile app development.



POJO



A POJO (Plain Old Java Object) is a simple Java class that represents the structure of data exchanged between the Android app and a RESTful API.

POJOs are used for data serialization and deserialization by Retrofit to convert JSON responses into Java objects and vice versa.





<https://jsonplaceholder.typicode.com>



<https://jsonplaceholder.typicode.com/albums>



HTTP QUERY



https://jsonplaceholder.typicode.com **/albums**

main url

endpoint



JSON SYNTAX



```
[{"key1": "value1"}, {"key2": "value2"}]
```

- [] - Array 
- {} - Objects
- Key
- Value
- : - Separates key and value.
- , - Separates objects.

Example:

```
[{"id": 1,  
  "name": "android"},  
 {"id": 2,  
  "name": "knowledge"}]
```



RETROFIT PROJECT



- **Step 1: Add Dependency**
- **Step 2: Add Permissions**
- **Step 3: Create a Data Class (POJO)**
- **Step 4: Create an Interface**
- **Step 5 : Setup Retrofit Instance**
- **Step 6: Receive Request**
- **Step 7: Display Data**